Module 3: Packages, Exception Handling and Multithreading

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai Campus

premanand.s@vit.ac.in

March 9, 2026

What is a Sub-Package?

- A sub-package is a package defined inside another package using dot notation.
- Example:

Example

```
package com.company.project;
```

- Java allows hierarchical naming using dots (.)
- Each level represents a folder in directory structure

Important: Sub-packages are NOT automatically related in terms of access control.

Folder Structure Representation

Package Declaration

```
package com.company.project;
```

Corresponding Directory Structure:

```
com/
|
+-- company/
|
+-- project/
```

- JVM class loader searches using this folder hierarchy
- Package name must match directory structure exactly

Example: Creating a Sub-Package

File: com/company/project/Employee.java

```
package com.company.project;

public class Employee {
 public void display() {
  System.out.println("Employee Class");
 }
}
```

Accessing a Sub-Package Class

File: MainApp.java

```
import com.company.project.Employee;

public class MainApp {
 public static void main(String[] args) {
  Employee e = new Employee();
  e.display();
 }
}
```

- Must import using full package path
- Folder hierarchy must exist before compilation

Very Important Concept (Common Misconception)

Question:

If we have:

- `package com.company;`
- `package com.company.project;`

Are they considered the same package?

Very Important Concept (Common Misconception)

Question:
If we have:

- `package com.company;`
- `package com.company.project;`

Are they considered the same package?
Answer: No.

Very Important Concept (Common Misconception)

Question:
If we have:

- `package com.company;`
- `package com.company.project;`

Are they considered the same package?
Answer: No.

- Sub-package is a completely separate package
- Default access does NOT work between them

Misconception Demo Code

File: com/company/A.java

```
package com.company;
public class A {
 int x = 100;   // defaultaccess  }
```

File: com/company/project/B.java

```
package com.company.project;
import com.company.A;
public class B {
 public static void main(String[] args) {
  A obj = new A();
  System.out.println(obj.x);  // Will this work?
 }  }
```

Misconception Demo Code

File: com/company/A.java

```
package com.company;
public class A {
 int x = 100;   // defaultaccess  }
```

File: com/company/project/B.java

```
package com.company.project;
import com.company.A;
public class B {
 public static void main(String[] args) {
  A obj = new A();
  System.out.println(obj.x);  // Will this work?
 }  }
```

Answer: Error (Different package)

Why Java Uses Hierarchical Packages

- Avoid global naming conflicts
- Support large-scale enterprise applications
- Enable modular project organization
- Align with company domain names

Reverse Domain Naming Convention:

- com.google
- org.apache
- edu.university

Compilation Using Command Prompt

```
javac -d . com/company/project/Employee.java
javac MainApp.java
java MainApp
```

- -d . creates proper directory structure
- Always compile from project root folder

Why Do We Need Import Statements?

- Java programs often use classes from other packages
- Import statements allow us to use those classes easily
- Without import, we must use the fully qualified class name

Example

```
java.util.Scanner
```

- Instead of writing the full name every time, we use import

Importing a Specific Class

Syntax

```
import package_name.ClassName;
```

Example

```
import java.util.Scanner;

public class Test {
 public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);
 }
}
```

- Only the specified class is available

Importing All Classes in a Package

Syntax

```
import package_name.*;
```

Example

```
import java.util.*;

public class Example {
 public static void main(String[] args) {
  ArrayList list = new ArrayList();
  Scanner sc = new Scanner(System.in);
 }
}
```

- Allows access to all public classes in that package

Important Clarification

Common Misconception

Important Clarification

Common Misconception

- `import java.util.*;` does NOT load all classes into memory

Important Clarification

Common Misconception

- `import java.util.*;` does NOT load all classes into memory
- The compiler only loads classes when they are actually used

Important Clarification

Common Misconception

- `import java.util.*;` does NOT load all classes into memory
- The compiler only loads classes when they are actually used
- It simply tells the compiler where to search for class definitions

Using Fully Qualified Class Name

Without Import Statement

```
public class Demo {
 public static void main(String[] args) {
  java.util.Scanner sc =
  new java.util.Scanner(System.in);
 }
}
```

- Full package path must be specified
- Import statements make code shorter and readable

Name Conflict Problem

Two packages may contain classes with the same name.

```
import java.util.Date;
import java.sql.Date;
```

Name Conflict Problem

Two packages may contain classes with the same name.

```
import java.util.Date;
import java.sql.Date;
```

Question: Which Date class should Java use?

Name Conflict Problem

Two packages may contain classes with the same name.

```
import java.util.Date;
import java.sql.Date;
```

Question: Which Date class should Java use?
Answer: Compiler error due to ambiguity.

Resolving Name Conflicts

Use the fully qualified class name.

```
import java.util.Date;

public class Test {
 public static void main(String[] args) {

  java.util.Date d1 =
  new java.util.Date();

  java.sql.Date d2 =
  new java.sql.Date(0);
 }
}
```

- Fully qualified names remove ambiguity

Import Variations Summary

Import Type	Example
Specific class import	import java.util.Scanner;
All classes in package	import java.util.*;
Fully qualified name	java.util.Scanner

Key Takeaways

- Sub-packages are independent packages in Java
- Folder structure must match the package declaration
- Default access works only within the same package
- Protected allows subclass access across packages
- Hierarchical naming supports scalable software design

Memory Rule:

$$\text{Package hierarchy} \neq \text{Access hierarchy}$$

From Packages to Exception Handling

What We Learned So Far

- Organizing classes using **packages**
- Creating and accessing packages
- Sub-packages and hierarchical structure
- Access modifiers across packages
- Import statements and name conflict resolution

From Packages to Exception Handling

What We Learned So Far

- Organizing classes using **packages**
- Creating and accessing packages
- Sub-packages and hierarchical structure
- Access modifiers across packages
- Import statements and name conflict resolution

But a question remains...

From Packages to Exception Handling

What We Learned So Far

- Organizing classes using **packages**
- Creating and accessing packages
- Sub-packages and hierarchical structure
- Access modifiers across packages
- Import statements and name conflict resolution

But a question remains...

What happens when something goes wrong during program execution?

- File not found
- Invalid user input
- Division by zero
- Network failure

From Packages to Exception Handling

What We Learned So Far

- Organizing classes using **packages**
- Creating and accessing packages
- Sub-packages and hierarchical structure
- Access modifiers across packages
- Import statements and name conflict resolution

But a question remains...

What happens when something goes wrong during program execution?

- File not found
- Invalid user input
- Division by zero
- Network failure

Java provides a mechanism to handle such situations:

A Simple Situation

Consider the following program

```
int a = 10;
int b = 0;
int result = a / b;
System.out.println(result);
```

A Simple Situation

Consider the following program

```
int a = 10;
int b = 0;
int result = a / b;
System.out.println(result);
```

What will happen when this program runs?

A Simple Situation

Consider the following program

```
int a = 10;
int b = 0;
int result = a / b;
System.out.println(result);
```

What will happen when this program runs?

- The program crashes
- JVM stops execution
- Remaining code does not execute

A Simple Situation

Consider the following program

```
int a = 10;
int b = 0;
int result = a / b;
System.out.println(result);
```

What will happen when this program runs?

- The program crashes
- JVM stops execution
- Remaining code does not execute

Problem

Programs may fail during execution due to unexpected situations.

What is an Exception?

- An **Exception** is an abnormal condition that occurs during program execution.

What is an Exception?

- An **Exception** is an abnormal condition that occurs during program execution.
- It interrupts the normal flow of the program.

What is an Exception?

- An **Exception** is an abnormal condition that occurs during program execution.
- It interrupts the normal flow of the program.
- Exceptions occur at **runtime**.

Examples

- Division by zero
- File not found
- Invalid array index
- Invalid user input

Why Do We Need Exception Handling?

Without exception handling:

- Program terminates abruptly
- Remaining code does not execute
- Poor user experience
- Difficult debugging

Why Do We Need Exception Handling?

Without exception handling:

- Program terminates abruptly
- Remaining code does not execute
- Poor user experience
- Difficult debugging

With exception handling:

- Program handles errors gracefully
- Execution continues
- Error messages can be displayed
- Improves reliability

Example Without Exception Handling

```
public class Test {
 public static void main(String[] args) {

  int a = 10;
  int b = 0;

  int result = a / b;

  System.out.println(result);
  System.out.println("Program Finished");
 }
}
```

Example Without Exception Handling

```
public class Test {
 public static void main(String[] args) {

  int a = 10;
  int b = 0;

  int result = a / b;

  System.out.println(result);
  System.out.println("Program Finished");
 }
}
```

Output

```
Exception in thread "main"
java.lang.ArithmeticException
```

Handling the Exception

```
public class Test {
 public static void main(String[] args) {

  int a = 10;
  int b = 0;

  try {
   int result = a / b;
   System.out.println(result);
  }

  catch (ArithmeticException e) {
   System.out.println("Cannot divide by zero");
  }
  System.out.println("Program continues...");
 }  }
```

Exception Handling Flow

1. Program starts execution

Exception Handling Flow

1. Program starts execution
2. Error occurs inside try block

Exception Handling Flow

1. Program starts execution
2. Error occurs inside try block
3. JVM creates an exception object

Exception Handling Flow

1. Program starts execution
2. Error occurs inside try block
3. JVM creates an exception object
4. Control moves to catch block

Exception Handling Flow

1. Program starts execution
2. Error occurs inside try block
3. JVM creates an exception object
4. Control moves to catch block
5. Error is handled

Exception Handling Flow

1. Program starts execution
2. Error occurs inside try block
3. JVM creates an exception object
4. Control moves to catch block
5. Error is handled
6. Program continues execution

Structure of Exception Handling

```
try {

 // risky code

}

catch(ExceptionType e) {

 // handling code

}

finally {

 // always executes
}
```

Finally Block

Purpose

- Code that must always execute

Finally Block

Purpose

- Code that must always execute
- Used for resource cleanup

Finally Block

Purpose

- Code that must always execute
- Used for resource cleanup
- Examples:
  - Closing files
  - Closing database connections

User Defined Exception

```
class InvalidAgeException extends Exception {

 InvalidAgeException(String message) {
  super(message);
 }
}

public class Test {
 static void checkAge(int age)
 throws InvalidAgeException {

  if(age < 18)
  throw new InvalidAgeException(
  "Not eligible");
 }
}
```

Using throw

Purpose:

- Used to explicitly throw an exception
- Used inside a method or block

Example

```
public class Test {
 public static void main(String[] args) {
  int age = 15;
  if(age < 18){
   throw new ArithmeticException(
   "Not eligible to vote");
  }
  System.out.println("Eligible to vote");
 }  }
```

Using throw

Purpose:

- Used to explicitly throw an exception
- Used inside a method or block

Example

```
public class Test {
 public static void main(String[] args) {
  int age = 15;
  if(age < 18){
   throw new ArithmeticException(
   "Not eligible to vote");
  }
  System.out.println("Eligible to vote");
 }  }
```

- If age is less than 18, program throws an exception
- JVM stops normal execution

Using throws

Purpose:

- Used to declare exceptions
- Written in method signature

Example

```
import java.io.*;
public class Test {
 static void readFile() throws IOException {
  FileReader file =
  new FileReader("data.txt");
 }
 public static void main(String[] args) {
  try{
   readFile();
  }
  catch(IOException e){
   System.out.println("File not found");
  }  }  }
```

throw vs throws

throw	**throws**
Used to throw exception	Used to declare exception
Used inside method	Used in method signature
Throws one exception	Can declare multiple exceptions

User Defined Exceptions

- Java allows programmers to create their own exceptions.

User Defined Exceptions

- Java allows programmers to create their own exceptions.
- These are called **User Defined Exceptions** or **Custom Exceptions**.

User Defined Exceptions

- Java allows programmers to create their own exceptions.
- These are called **User Defined Exceptions** or **Custom Exceptions**.
- They are created by extending the **Exception** class.

Syntax of User Defined Exception

```
class MyException extends Exception {

 MyException(String message) {
  super(message);
 }

}
```

- Custom exception class extends Exception
- Constructor passes message to parent class

Example: Invalid Age Exception

```
class InvalidAgeException extends Exception {
 InvalidAgeException(String msg){
  super(msg);
 }  }
public class Test {
 static void checkAge(int age)
 throws InvalidAgeException {
  if(age < 18)
  throw new InvalidAgeException("Not eligible to vote");
  System.out.println("Eligible to vote");   }
 public static void main(String[] args) {
  try{
   checkAge(16);    }
  catch(InvalidAgeException e){
   System.out.println(e.getMessage());
  }    }   }
```

Handling Custom Exception

```
public class Main {

 public static void main(String[] args) {

  try{
   Test.checkAge(16);
  }

  catch(InvalidAgeException e){
   System.out.println(e.getMessage());
  }

 }
}
```

Thank You!

Stay Connected

Premanand S

Email:	premanand.s@vit.ac.in
Phone:	+91-7358679961
LinkedIn:	linkedin.com/in/premsanand
Instagram:	instagram.com/premsanand
WhatsApp Channel:	anandsDataX
Google Scholar:	Google Scholar Profile
GitHub:	github.com/anandprems